

哈尔滨工业大学

硕士学位论文中期报告

题 目：移位异或纠错码的高效实现研究

院 (系) 计算机科学与技术学院

学 科 计算机科学与技术

导 师 王强副教授

研 究 生 孙铎

学 号 24S051046

中期报告日期 2026 年 6 月 15 日

目 录

1	课题主要研究内容及进度	1
1.1	课题主要研究内容	1
1.2	进度介绍	2
2	目前已完成的主要研究工作及结果	2
2.1	Two-tone 移位异或纠删码的原理分析	3
2.1.1	移位异或编码原理	3
2.1.2	移位异或解码原理	4
2.1.3	直观实现的性能瓶颈	6
2.2	系统级优化方案的设计与实现	6
2.2.1	统一解码抽象: Erasure-Parity Mapping	7
2.2.2	缓存友好的窗口划分: Cache-Friendly Window Partitioning	9
2.2.3	内存友好的相邻合并访问: Neighbor-Merging Access	10
2.2.4	遍历模式选择: Traversal Pattern Selection	11
2.3	基于 Ceph 的系统集成	13
2.4	实验评估与结果	13
2.4.1	实验环境与配置	13
2.4.2	编码与解码吞吐	13
2.4.3	消融实验	15
2.4.4	端到端存储系统实验	16
3	后期拟完成的研究工作及进度安排	16
3.1	第一点: 已有系统优化的机理验证与实验深化	17
3.2	第二点: 小规模覆盖写下的 Two-tone/FastTT 增量更新	18
4	存在的困难与问题	19
4.1	存在问题与困难	19
4.2	解决方案	20
5	如期完成全部论文工作的可能性	20

1 课题主要研究内容及进度

1.1 课题主要研究内容

在现代大规模分布式存储系统中，数据可用性是应对软硬件故障的基本保障^[1]。当前主流的容错技术包括数据复制和纠删码：数据复制实现简单，但存储开销大（如三副本需要三倍存储空间）；而纠删码通过对原始数据分块编码生成冗余校验信息，能够以远低于复制的存储成本提供同等甚至更高的数据可靠性^[2]。正因如此，纠删码已成为 Google 文件系统（GFS）^[3]、Windows Azure 存储^[4]、Hadoop 分布式文件系统（HDFS）^[5] 以及 Ceph^[6] 等大规模存储系统的核心技术。

经典的 Reed-Solomon（RS）码^[7] 基于有限域 $GF[2^w]$ 上的矩阵乘法进行编解码，具有良好的最大距离可分（MDS）特性。为提升其性能，研究者将部分有限域运算转化为异或运算^[8]，并在矩阵稀疏化、公共子表达式复用、内存与缓存访问优化、指针开销削减等系统层面持续改进，代表性工作包括 Jerasure^[9]、G-CRS^[10]、Zerasure^[11]、SLPEC^[12]、Cerasure^[13] 以及 Intel 的 ISA-L 库^[14]。近年来，基于非循环移位异或（Shift-XOR）运算的新型纠删码逐渐成为理论研究热点。ZigZag Decodable Codes^[15] 通过构建特殊的移位矩阵，仅依赖移位和异或操作即可实现高效编解码；Two-tone Shift-XOR Codes^[16] 在连续可解性（Successive Solvability）理论^[17] 的基础上进一步规范了移位矩阵结构，设计了先递减后递增的对称偏移模式，是目前移位异或纠删码的 SOTA 成果。对这类编码而言，异或就是最基本的运算，无需像 RS 码那样进行有限域转换，因而具备更高的理论性能潜力。

然而，尽管 Two-tone 移位异或纠删码在理论上性能优越，其完整的工程实现与系统级优化仍然缺失。现有文献仅在实验测试中验证了算法潜力，缺乏可用于生产环境的高性能实现，具体表现为三方面问题：

- 1) **缺少泛化性实现**：现有实现无法应对数据块和校验块的所有丢失情况，不能同时满足存储系统中“解码丢失数据块”与“重编码丢失校验块”的需求。
- 2) **缺少系统级优化**：理论算法未给出减少条件分支与遍历、增强缓存局部性、缓解内存访问压力等优化策略，而 RS 码的系统级优化方法因算法本质不同而无法直接迁移到移位异或纠删码上。
- 3) **缺少工业级应用**：移位异或纠删码尚未集成到 Ceph、Hadoop 等开源存储系统中，缺乏工业级场景下的高效性验证。

针对上述问题，本课题以 Two-tone 非循环移位异或纠删码为研究对象，研究其系统级高效实现方法，主要研究内容包括：（1）深入分析 Two-tone 移位异或纠删码的编解码原理，明确其在工程实现中的内存访问瓶颈与优化方向；（2）提出并实

现一套面向移位异或纠删码的系统级优化方案，包括统一解码抽象、缓存友好的窗口划分、内存友好的相邻合并访问以及面向常见丢失场景的特化优化；（3）将优化后的 Two-tone 纠删码以插件形式集成至 Ceph 分布式存储系统，验证其功能完整性与工业级部署能力；（4）通过消融实验与对比实验，验证各项优化的有效性，并与 SOTA 纠删码（ISA-L、Cerasure 等）进行性能对比，最终提炼出针对移位异或纠删码的系统优化方法论，填补该领域的空白。

1.2 进度介绍

截至中期，本课题已完成开题报告中规划的绝大部分核心研究工作，并取得了显著的阶段性成果。相关研究成果已整理形成一篇学术论文，投稿至并行与分布式处理国际会议（IPDPS 2026）。总体进度如下：

- 1) **理论分析与方案设计（已完成）**：完成了纠删码相关理论与系统优化方法的文献调研，深入剖析了 Two-tone 移位异或纠删码的编解码原理，明确了其“内存受限（memory-bound）”的本质瓶颈，并据此设计了四项系统级优化方案。
- 2) **高效实现与系统集成（已完成）**：使用约 2000 行 C++ 代码实现了名为 FastTT 的高性能 Two-tone 纠删码库，集成了全部四项优化，并以插件形式集成至 Ceph v18.2.7 存储系统，成功通过了 Ceph 纠删码接口测试与端到端集群读写测试。
- 3) **实验评估（已完成）**：在多种 (k, m) 配置下完成了编码吞吐、解码吞吐、消融实验以及基于磁盘的端到端吞吐实验，验证了各项优化的有效性，并相较 ISA-L、Jerasure、Cerasure 等 SOTA 实现取得了显著的性能优势。
- 4) **方法论总结与论文撰写（进行中）**：已系统性总结移位异或纠删码的系统优化方法论并完成学术论文撰写与投稿，后续将持续完善实验与论文，并撰写毕业论文。

2 目前已完成的主要研究工作及结果

本部分详细介绍中期阶段已完成的主要研究工作，包括移位异或纠删码的原理分析、四项系统级优化方案的设计与实现、Ceph 系统集成，以及完整的实验评估结果。

2.1 Two-tone 移位异或纠删码的原理分析

2.1.1 移位异或编码原理

移位异或纠删码采用系统化 $(k, k + m)$ 编码。原始消息被划分为 k 个数据块 $d_1, d_2, \dots, d_k$ ，每块含 l 个码字 (symbol)，一个码字通常设为一个 SIMD 寄存器的长度 (如 256 位)。前 k 个编码块即为数据块本身，其余 m 个块 $c_1, \dots, c_m$ 为校验块。系统化移位异或码的生成矩阵可由偏移矩阵 $S = (s_{i,j})$ 刻画，其中每个 $s_{i,j}$ 为非负整数，表示移位偏移量。第 i 个校验块由(2-1)得到：

$$c_i[x] = \bigoplus_{j=1}^k d_j[x - s_{i,j}], \quad (2-1)$$

其中越界访问按零处理 (即 $x \leq 0$ 或 $x > l$ 时 $d_j[x] = 0$)。移位操作可通过地址偏移直接实现，几乎无额外开销，这正是移位异或码相较 RS 码的核心优势。

在各类移位异或码中，Two-tone 码^[16] 泛化了既有构造并成为 SOTA。其偏移矩阵采用先递减后递增的对称结构。以 $(k, m) = (4, 3)$ 为例，可取偏移矩阵(2-2)：

$$\begin{pmatrix} 3 & 2 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 \end{pmatrix}. \quad (2-2)$$

即每个数据块分别右移、不移或左移若干码字后异或得到校验块，编码过程如图2-1所示。

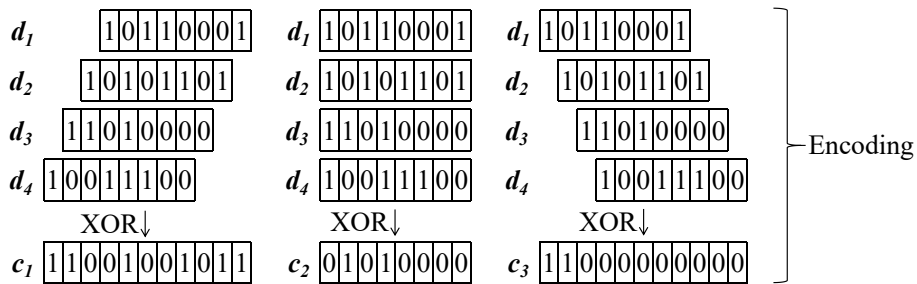


图 2-1 Two-tone 码编码举例 ($k = 4, m = 3$)

算法1给出了 Two-tone 编码的直观实现：按数据块逐码字扫描，并依据偏移矩阵 $(s_{i,j})$ 将其异或贡献累加到对应的校验块码字上。这种实现方式会使不同数据块码字分散且重复地写入相同的校验块码字，造成较差的局部性与冗余的内存访问。

Algorithm 1: TWO-TONE ENCODING

Input: Data chunks $d_1, d_2, \dots, d_k$, each of length l , Two-tone offset matrix $(s_{i,j})$.
Output: Parity chunks $c_1, c_2, \dots, c_m$.

```

1 for  $j \leftarrow 1$  to  $k$  do
2   for  $x \leftarrow 1$  to  $l$  do
3     for  $i \leftarrow 1$  to  $m$  do
4        $c_i[x + s_{i,j}] \leftarrow c_i[x + s_{i,j}] \oplus d_j[x]$ ;
5     end
6   end
7 end
    
```

2.1.2 移位异或解码原理

得益于高度规则的 Two-tone 构造，解码可按“连续可解”^[17]的顺序进行，每个码字的求解顺序可预先确定，无需暴力搜索。沿用前述 $(k, m) = (4, 3)$ 的例子，假设丢失了数据块 d_1, d_2 以及校验块 c_2 ，存活的数据块为 d_3, d_4 、存活的校验块为 c_1, c_3 ，完整的解码与 c_2 修复过程如图2-2所示。

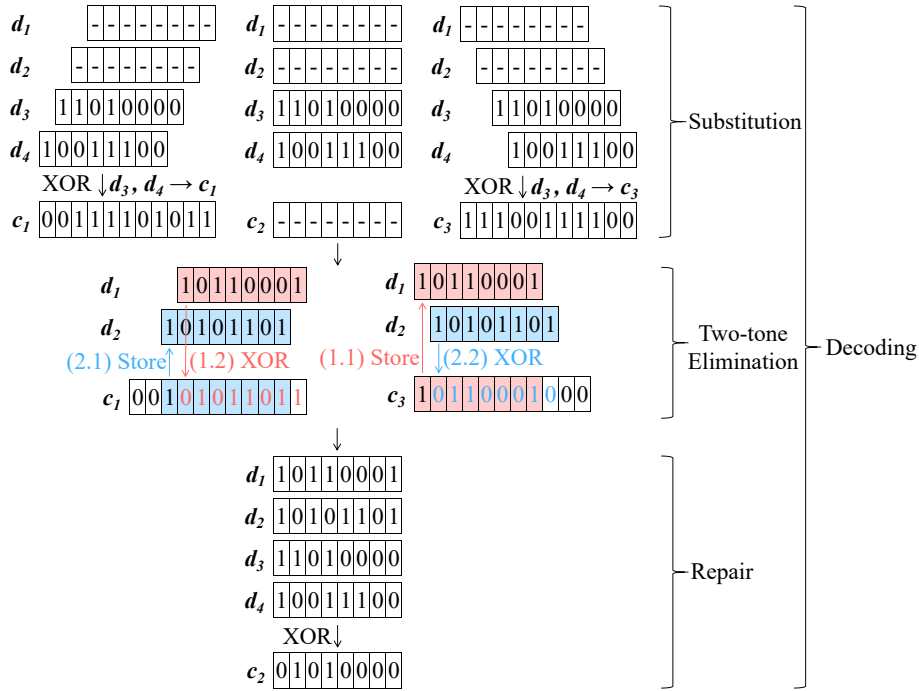


图 2-2 Two-tone 码解码举例（假设 d_1, d_2, c_2 丢失）

解码总体分为三个阶段。第一阶段“代入（Substitution）”：将存活的数据块 d_3, d_4 按偏移异或代入存活的校验块 c_1, c_3 ，从而消去已知数据的贡献，使与丢失块相关的码字暴露出来。第二阶段“Two-tone 消除（Elimination）”：从两侧最外侧错开的码字开始逐个求解丢失数据块。具体地， d_1, d_2 的首个码字可直接得到；将

$d_1[1]$ 、 $d_2[1]$ 分别代入 c_3 、 c_1 后，可得 $d_1[2] = c_3[2] \oplus d_2[1]$ 、 $d_2[2] = c_1[4] \oplus d_1[1]$ ；再将 $d_1[2]$ 、 $d_2[2]$ 代入，又得 $d_1[3] = c_3[3] \oplus d_2[2]$ 、 $d_2[3] = c_1[5] \oplus d_1[2]$ ；如此交替推进，直至顺序解出全部码字。这里的“两侧”恰好对应 Two-tone 算法的两个子系统，分别对应编码时递减与递增的部分。第三阶段“修复 (Repair)”：由于校验块 c_2 也已丢失，需在数据块全部恢复后重新编码以修复之；因偏移矩阵中 $s_{2,j} = 0$ ($1 \leq j \leq 4$)，有 $c_2 = d_1 \oplus d_2 \oplus d_3 \oplus d_4$ 。

算法2给出了上述解码流程的直观实现，包含代入、Two-tone 消除与修复三个逻辑阶段。代入阶段与编码同理，会带来相同的冗余访问问题；消除阶段以一个同步偏移 Δ 对齐不同的移位量，并围绕中心索引 j_{mid} 将存活校验块划分为两个子系统，从两端向中间确定性地依次暴露并求解码字；最后再对丢失的校验块单独执行一次修复。可以看到，直观实现将丢失数据块的解码与丢失校验块的修复分离处理，且对所有丢失情况默认采用以数据块为中心的同一种遍历模式。

Algorithm 2: TWO-TONE DECODING

Input: Ascendingly sorted indices of survived parity chunks I_{svd} , indices of erased parity chunks I_{erd} , indices of survived data chunks J_{svd} , indices of erased data chunks J_{erd} , survived data chunks $d_j, j \in J_{svd}$, survived parity chunks $c_i, i \in I_{svd}$, Two-tone offset matrix ($s_{i,j}$)

Output: Decoded data chunks $d_j, j \in J_{erd}$, repaired parity chunks $c_i, i \in I_{erd}$

/* Line 1-7: *Substitution*($d[J_{svd}] \rightarrow c[I_{svd}]$) */

```

1  foreach  $j \in J_{svd}$  do
2      for  $x \leftarrow 1$  to  $l$  do
3          foreach  $i \in I_{svd}$  do
4               $c_i[x + s_{i,j}] \leftarrow c_i[x + s_{i,j}] \oplus d_j[x]$ ;
5          end
6      end
7  end

```

/* Line 8-17: *Elimination*($d[J_{erd}] \leftrightarrow c[I_{svd}]$) */

```

8   $j_{mid} \leftarrow \underset{j}{\operatorname{argmin}} |I_{svd}[j] - 1 - m/2|$ ;
9  for  $x \leftarrow \min(2 - m/2, 1)$  to  $l$  do
10     for ( $j_{idx} \leftarrow |J_{erd}|$  to  $j_{mid} + 1$ )
11         then ( $j_{idx} \leftarrow 1$  to  $j_{mid}$ ) do
12              $j \leftarrow J_{erd}[j_{idx}]$ ;  $i \leftarrow I_{svd}[j_{idx}]$ ;
13              $\Delta \leftarrow \max(|i - 1 - m/2| - 1, 0)$ ;
14              $d_j[x + \Delta] \leftarrow c_i[x + \Delta + s_{i,j}]$ ;
15             Substitute( $d_j[x + \Delta] \rightarrow c[I_{svd} - i]$ );
16         end
17     end
18  Repair  $c[I_{erd}]$ ;

```

2.1.3 直观实现的性能瓶颈

对上述算法的直观实现进行分析可知，其性能受限于内存访问，主要存在三方面瓶颈：

- 1) **冗余内存访问与较差的时间局部性**：编码时按数据块顺序逐码字扫描，每次更新都需访问全部校验块，导致校验块被反复读写，时间局部性差；解码的代入与消除阶段存在同样问题，使整体实现成为内存受限型。
- 2) **低效的数据遍历模式**：编码可采用横向或纵向两种遍历模式。横向遍历对数据块利用充分，但对校验块反复写入；纵向遍历减少了校验块写入，却需重复加载数据块。两者各有内存访问瓶颈。
- 3) **复杂的解码流程与次优遍历模式**：多数已有工作只重视数据块的及时修复而对校验块采取惰性策略，但校验块同样关键——若校验块不及时修复，再发生一次数据块丢失就可能造成数据永久不可用。此外，对所有丢失情况采用统一的遍历模式也会导致次优性能。

针对上述瓶颈，本课题提出四项系统级优化技术并整合进 FastTT 实现。

2.2 系统级优化方案的设计与实现

针对上述内存访问瓶颈，本课题提出了四项相互配合的系统级优化技术，并将其整合进 FastTT 实现中。图2-3给出了 FastTT 的整体优化框架：编码阶段先按缓存友好窗口划分（Cache-Friendly Window Partitioning）将数据块切分为小窗口，并以相邻合并访问（Neighbor-Merging Access）模式计算移位异或结果写入校验块；解码阶段先通过统一解码抽象（Erasure-Parity Mapping）统一数据块与校验块的丢失情况，再依据丢失情况进行遍历模式选择（Traversal Pattern Selection），最后同样以窗口划分方式完成代入与 Two-tone 消除，一并完成丢失数据块的解码与丢失校验块的修复。

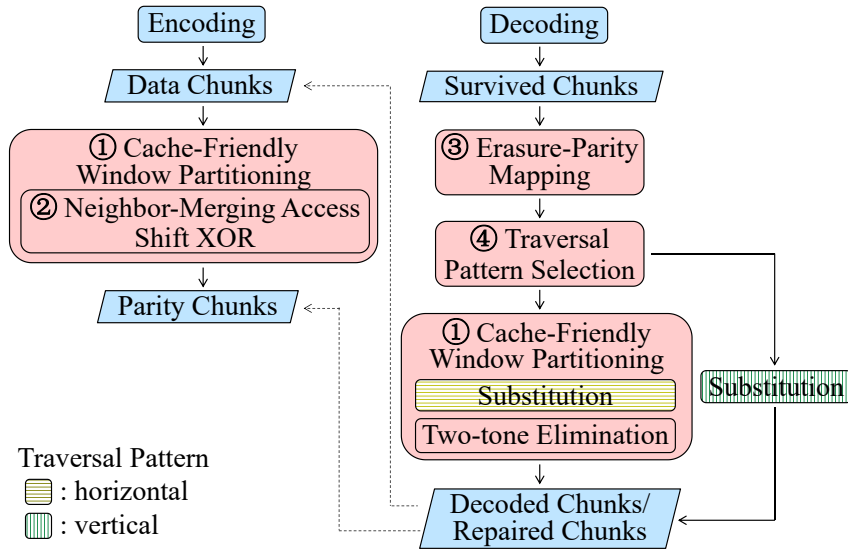


图 2-3 FastTT 的整体优化框架

2.2.1 统一解码抽象：Erasure-Parity Mapping

在通用的存储场景中，解码必须同时保证数据块和校验块的及时恢复。直观实现需要在解码完成后单独执行一个“修复（Repair）”阶段来重新编码丢失的校验块，引入了冗余操作和额外的内存访问。为此，本课题提出 Erasure-Parity Mapping 统一抽象：将所有丢失的数据块和丢失的校验块统一表示为同一种映射结构，每个丢失块按编号顺序匹配一个未丢失的校验块；若丢失块本身就是校验块，则将其映射至自身。在代入步骤中无需额外判断，直接将未丢失数据块代入所有映射结构；在消除步骤中，每解出一个码字便立即代入到所有相关映射结构（包括对应丢失校验块的结构）中。由于数据块的解码与校验块的重编码被统一为对同类映射结构的操作，二者得以相互穿插、同时完成，从而省去了独立的修复阶段，进一步减少了遍历与内存访问次数。图2-4与图2-5对比了一般解码流程与本方案的统一解码流程。两图右侧标注了各阶段的复杂度，其中 MEM 表示内存访问操作的次数，XOR 表示异或操作的次数。对比可见，两种流程的异或操作次数总体相当——这是因为统一解码并未改变底层的 Two-tone 消除算法逻辑，求解每个丢失块所需的异或运算量是固定的；真正的差异体现在内存访问上：一般流程需要额外的校验块复制（图2-4中的“Copy parity chunks”）以及独立的校验块重编码（“Re-encode erased parity chunks”）阶段，二者都引入了大量额外的 MEM 操作；而统一解码流程通过 Erasure-Parity Mapping 将校验块修复融入消除步骤，省去了上述独立阶段，从而在不增加异或运算的前提下显著减少了内存访问的总次数。

图2-6给出了一个具体的解码示例。假设 d_1 、 d_2 和 c_2 丢失，映射步骤建立三组

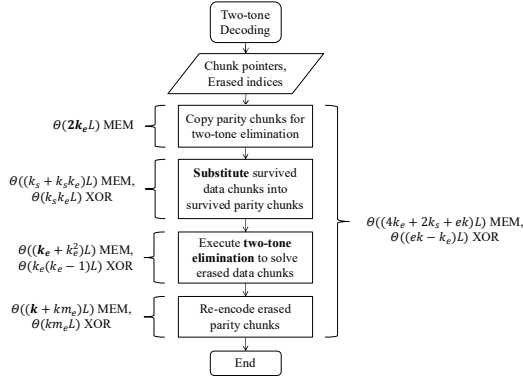


图 2-4 Two-tone 一般解码实现流程

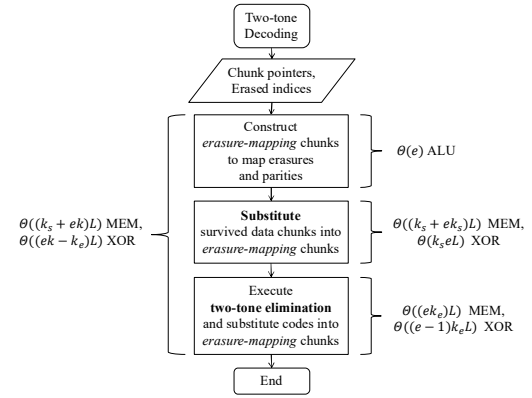
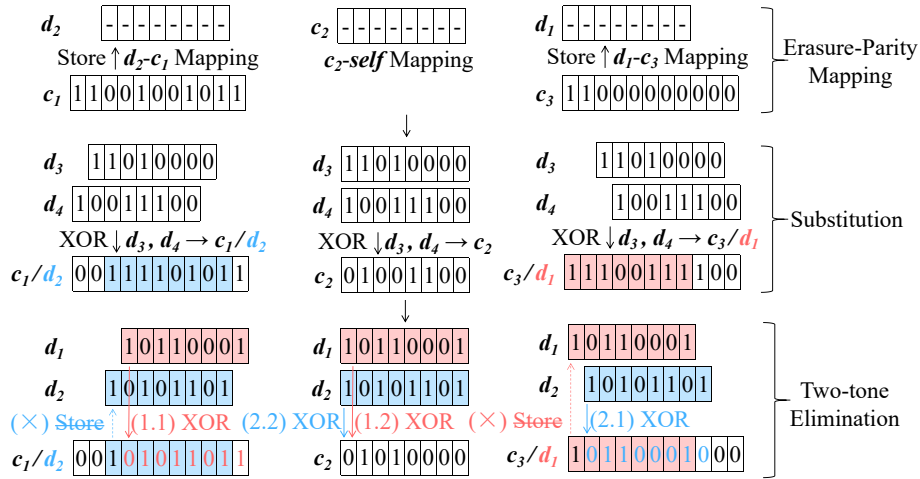


图 2-5 Erasure-Parity Mapping 统一解码流程

关联： $d_1 \leftrightarrow c_3$ 、 $c_2 \leftrightarrow c_2$ （自映射）、 $d_2 \leftrightarrow c_1$ ；随后将未丢失的数据块 d_3 、 d_4 代入这些映射；在消除过程中，解出的码字按序传播至其他映射结构，丢失校验块 c_2 的修复也被自然融入消除步骤，从而显著减少冗余内存访问。


 图 2-6 Erasure-Parity Mapping 解码举例（假设 d_1 、 d_2 、 c_2 丢失，虚线箭头与删除线标注表示被消除的冗余操作）

算法3给出了基于 Erasure-Parity Mapping 抽象的解码过程，包含映射、代入与 Two-tone 消除三个步骤。映射步骤将所有丢失的数据块与校验块统一表示为结构 mpe，记录每个丢失块及其匹配的存活校验块；若丢失块本身为校验块，则映射至自身。代入与消除步骤随后对所有 mpe 条目按统一规则处理，从而省去独立的修复阶段。

Algorithm 3: ERASURE-PARITY MAPPING DECODING

Input: Ascendingly sorted indices of survived parity chunks I_{svd} , indices of erased parity chunks I_{erd} , indices of survived data chunks J_{svd} , indices of erased data chunks J_{erd} , survived data chunks $\mathbf{d}_j, j \in J_{svd}$, survived parity chunks $\mathbf{c}_i, i \in I_{svd}$, Two-tone offset matrix $(s_{i,j})$

Output: Decoded data chunks $\mathbf{d}_j, j \in J_{erd}$, repaired parity chunks $\mathbf{c}_i, i \in I_{erd}$

```

/* Line 1-10: MapErasure( $J_{erd}|I_{erd} \leftrightarrow I_{svd}$ ) */
1  $mpe \leftarrow \text{allocate}(m, \text{nullptr}); I_{mpd} \leftarrow \text{vector}();$ 
2 foreach  $j \in J_{erd}$  do
3    $i \leftarrow I_{svd}.\text{pop\_front}(); I_{mpd}.\text{push\_back}(i);$ 
4    $mpe[i] \leftarrow \mathbf{d}_j; // \mathbf{d}_j[x] \text{ is undefined}$ 
5    $\text{memcpy}(\mathbf{c}_i \rightarrow mpe[i], m \times l);$ 
6 end
7 foreach  $i \in I_{erd}$  do
8    $mpe[i] \leftarrow \mathbf{c}_i; // \mathbf{c}_i[x] \text{ is undefined}$ 
9    $\text{memset}(0 \rightarrow mpe[i], m \times l);$ 
10 end
11  $\text{Substitution}(\mathbf{d}[J_{svd}] \rightarrow mpe[I_{mpd} + I_{erd}]);$ 
12  $\text{Elimination}(mpe[I_{mpd}] \leftrightarrow mpe[I_{mpd} + I_{erd}]);$ 
    
```

2.2.2 缓存友好的窗口划分：Cache-Friendly Window Partitioning

由于移位和异或操作本身极为高效，整体性能受限于数据块在内存中的读写效率，频繁的跨块内存访问极易导致缓存失效。为此，本课题将每个较长的数据块划分为连续、定长的窗口（window），并在处理下一个窗口前完成当前窗口内的全部移位异或操作，如图2-7所示，从而显著提升缓存命中率。同时，考虑到 Ceph 等系统分配的块内存通常未预先初始化，本方案采用“首次访问时初始化”（Init On First Access）策略，仅在窗口被实际访问时才进行初始化，避免了对全部块一次性初始化造成的开销与浪费。

在解码过程中，由于代入与消除两个步骤在不同块之间存在错位，若仅对当前窗口代入，可能导致消除位置的数据未被完全覆盖。为此本方案采用 off-by-one 代入策略：代入窗口始终比消除窗口领先一个窗口，从而保证代入覆盖消除将要读取的全部位置。该策略要求窗口大小至少能跨越 $k - 1$ 个码字位置，以覆盖 Two-tone 偏移矩阵引入的全部相对偏移；据此选取满足 $W \geq k - 1$ 的最小 2 的幂作为窗口大小 W ，兼顾对齐简洁性与覆盖要求，实验证明该取值能取得最佳的吞吐折中。

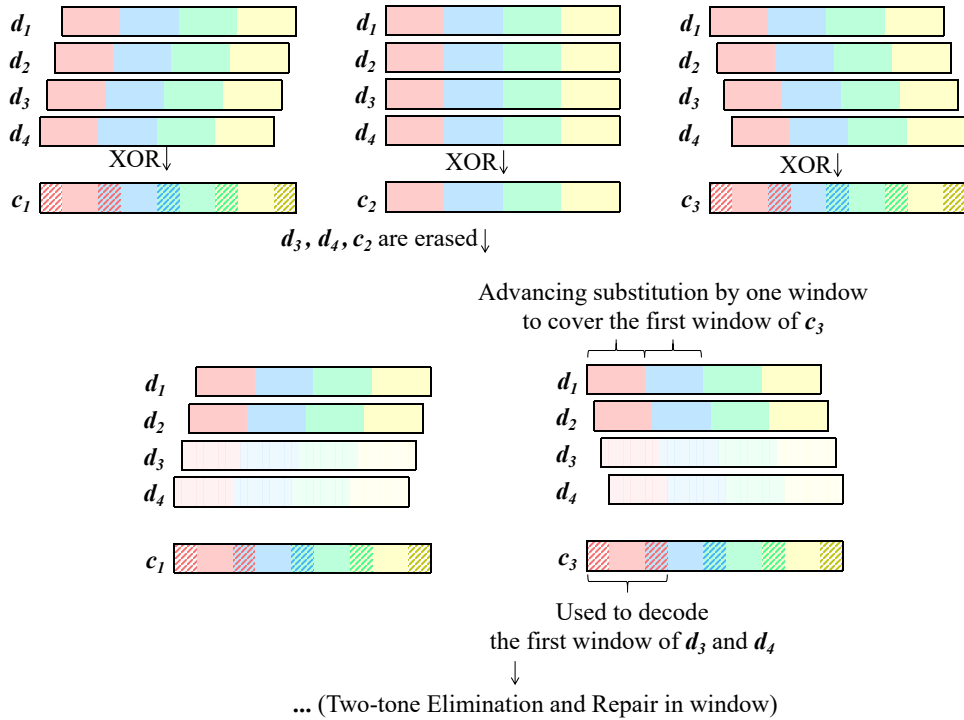


图 2-7 缓存友好窗口划分效果演示。同色片段表示同一窗口内的局部访问，校验块中的阴影区域表示与相邻窗口的重叠

2.2.3 内存友好的相邻合并访问：Neighbor-Merging Access

编码时存在横向与纵向两种遍历模式（如图2-8(a)、(b)所示）：横向遍历逐个读取数据块码字并立即更新所有对应校验块，数据访问局部性好，但校验块写入分散且重复，内存访问复杂度约为 $\Theta((k + km)l)$ ；纵向遍历以校验块码字为中心，一次性写入每个校验块码字，减少了校验块写入，却需重复加载数据块，复杂度约为 $\Theta((m + km)l)$ 。两者均存在内存访问瓶颈。

为此本课题提出相邻合并访问策略：在横向遍历的基础上，将相邻的两个数据块码字先加载到 SIMD 寄存器中，通过一次寄存器内异或合并，再一次性写入对应校验块，如图2-8(c)所示。相邻合并访问的内存访问复杂度降低为约 $\Theta((k + \frac{km}{2})l)$ ，明显优于横向遍历，并在 $m(k + 2) > 2k$ （实际 (k, m) 配置普遍满足）时也优于纵向遍历。该方法既避免了对同一数据块内存的重复访问，又将对校验块内存的访问理论上减少一半，同时通过指令级两块异或提升了 SIMD 通道吞吐利用率。考虑到 SIMD 寄存器数量有限且更多校验块会增加偏移复杂度，本方案有意将合并限制在相邻两块之间。

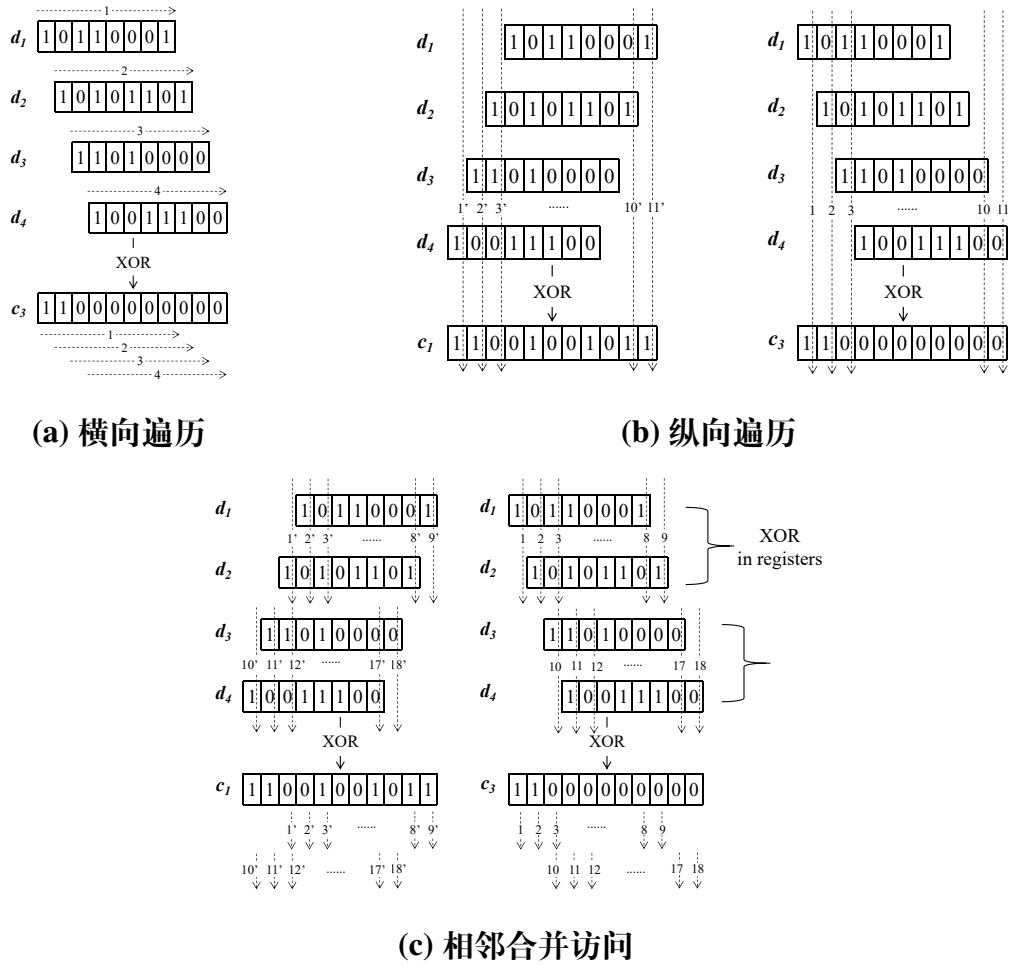


图 2-8 Two-tone 编码的访问模式对比：(a) 横向遍历，数据局部性好但校验写入分散；(b) 纵向遍历，校验写入集中但重复读取数据；(c) 相邻合并访问，相邻数据块窗口先在 SIMD 寄存器内异或合并，再以一次连续存储写回各校验块窗口

2.2.4 遍历模式选择：Traversal Pattern Selection

在大规模存储集群中，故障模式高度偏向单块丢失，同时丢失两块及以上的概率远低于丢失一块。为此本课题针对“仅丢失一个数据块或一个校验块”的常见场景设计了特化优化。此时解码无需执行完整的 Two-tone 消除流程，只需对所有相关的未丢失块做一次全局异或，一次性恢复丢失块。由于该操作对每个相关码字恰好访问一次，特别适合采用纵向遍历模式（此时无重复访问，效率高于横向模式），并可进一步采用 AVX2 的 stream 类指令让内存访问绕过缓存，省去缓存相关处理流程，如图2-9所示。该特化分支与统一解码框架完美兼容：解码器先执行 Erasure-Parity Mapping 构造映射，再根据丢失块数量选择遍历模式——丢失一块时走单遍纵向异或快路径，否则回退到标准的“代入 + 消除”流程。

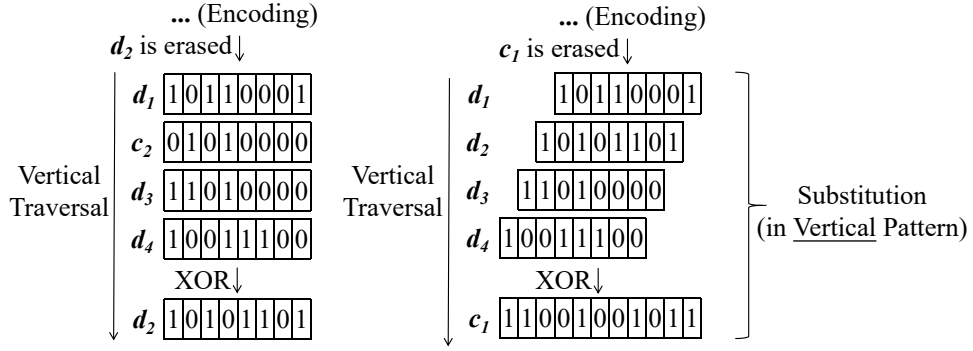


图 2-9 单块丢失场景下的纵向遍历模式演示：丢失块（如 d_2 或 c_1 ）通过对未丢失块的一遍全局异或一次性恢复，无需 Two-tone 消除

算法4给出了带遍历模式选择的统一解码过程。解码器先执行 Erasure-Parity Mapping 构造映射结构，再依据丢失块总数选择遍历模式：当恰好丢失一个块（数据块或校验块）时，跳过完整的 Two-tone 消除，仅以一遍纵向遍历对所有必要的存活块做全局异或、一次性重建丢失块；否则回退到标准的“代入 +Two-tone 消除”流程。

Algorithm 4: TRAVERSAL PATTERN SELECTION DECODING

Input: Ascendingly sorted indices of survived parity chunks I_{svd} , indices of erased parity chunks I_{erd} , indices of survived data chunks J_{svd} , indices of erased data chunks J_{erd} , survived data chunks $d_j, j \in J_{svd}$, survived parity chunks $c_i, i \in I_{svd}$, Two-tone offset matrix $(s_{i,j})$

Output: Decoded data chunks $d_j, j \in J_{erd}$, repaired parity chunks $c_i, i \in I_{erd}$

```

1 MapErasure( $J_{erd} \mid I_{erd} \leftrightarrow I_{svd}$ );
  /* Traversal Pattern Selection */
2 if  $|J_{erd}| + |I_{erd}| == 1$  then
  /* Vertical pattern reconstruction (single pass) */
3    $i \leftarrow I_{mpd}[1]$ ;
4   for  $x \leftarrow 1$  to  $l$  do
5      $sum \leftarrow 0$ ;
6     foreach  $j \in J_{svd}$  do
7        $sum \leftarrow sum \oplus d_j[x - s_{ij}]$ ;
8     end
9      $mpe[i][x] \leftarrow sum$ ;
10  end
11 end
12 else
  /* Standard unified decoding */
13    $Substitution(d[J_{svd}] \rightarrow mpe[I_{mpd} + I_{erd}])$ ;
14    $Elimination(mpe[I_{mpd}] \text{ from } mpe[I_{mpd} + I_{erd}])$ ;
15 end
    
```

2.3 基于 Ceph 的系统集成

为验证上述优化方案的工业级可用性，本课题将优化后的 Two-tone 纠删码以插件形式集成至 Ceph v18.2.7 分布式存储系统。集成工作分为两个层次：其一是接口集成与功能验证，通过 Ceph 标准纠删码接口对编解码的正确性与性能进行测试，借助 Erasure-Parity Mapping 机制确保算法能够应对所有可能的数据块与校验块丢失组合；其二是集群集成与应用验证，利用 Ceph 自带的 vstart 工具搭建测试集群，创建采用 Two-tone 编码的纠删码存储池，并完成文件的端到端读写。完整的功能性测试表明，FastTT 插件能够稳定可靠地在分布式环境中完成编解码与读写流程，具备工业级部署能力。

2.4 实验评估与结果

2.4.1 实验环境与配置

实验在配备 Intel Core i9-14900K（6.0 GHz，24 核，36 MiB L3 缓存）和 64 GB DDR5 内存的工作站上进行，操作系统为 Ubuntu 22.04，编译器为 GCC 11.4.0，并启用 AVX2 指令。对比基线包括三种代表性纠删码实现：Jerasure^[9]、Intel ISA-L^[14]（Ceph 官方支持、业界公认最高性能的生产级实现）以及学术界 SOTA 实现 Cerasure^[13]。实验在 (6, 2)、(8, 3)、(10, 4)、(6, 3) 四种代表性配置下进行，块大小固定为 2 MB，每组实验取重复 2000 次的平均吞吐。

2.4.2 编码与解码吞吐

在 Ceph 基准测试工具下，FastTT 的编码吞吐相较 ISA-L 平均提升 59.1%（最高 84.1%），相较 Jerasure 平均提升 261.4%（最高 342.8%），如表 2-1 所示。即便在 (10, 4) 这一相对不利的配置下（因第四个校验块需 2 码字移位、无法在 AVX2 256 位寄存器约束下合并），FastTT 仍优于两种基线。

表 2-1 FastTT、ISA-L 与 Jerasure 的编码吞吐 (GB/s)

配置	FastTT	ISA-L	Jerasure
(6, 2)	34.74	24.37	17.03
(6, 3)	31.69	17.59	7.64
(8, 3)	31.35	17.03	7.08
(10, 4)	17.59	13.56	4.58

在解码方面，FastTT 相较 ISA-L 平均提升 50.9%，相较 Jerasure 平均提升 161.1%。其中单块丢失场景的优势尤为突出，相较 ISA-L 提升超过 130%（最高 182.1%），这得益于绕过完整消除流程的直接异或快路径；对 Jerasure 的最大提升则出现在丢失三块或四块的高丢失场景（达 212.5%–328.8%）。表2-2给出了全部四种配置、各丢失块数下的解码吞吐（丢失块数列为同时丢失的块数）。需要说明的是，在两块丢失以及 (10,4) 配置下丢失多块时，FastTT 的优势相对较小、个别情形略低于 ISA-L：前者是因为两块丢失既无法利用单块快路径、又不具备高丢失场景下异或主导的特点，必须完整执行代入与 Two-tone 消除；后者则源于 (10,4) 第四个校验块的 2 码字移位扰乱了代入与消除流程（与编码端瓶颈同源）。即便如此，其解码吞吐仍与 ISA-L 接近，且编码性能始终更优。

表 2-2 FastTT、ISA-L 与 Jerasure 的解码吞吐（GB/s）

配置	丢失块数	FastTT	ISA-L	Jerasure
(6, 2)	1	95.99	38.38	42.70
(6, 2)	2	25.60	24.89	16.26
(6, 3)	1	95.18	39.43	44.57
(6, 3)	2	24.24	24.77	12.12
(6, 3)	3	19.18	18.16	5.39
(8, 3)	1	93.11	33.01	37.04
(8, 3)	2	26.18	23.16	10.97
(8, 3)	3	21.44	18.38	5.00
(10, 4)	1	81.01	34.42	35.11
(10, 4)	2	20.96	22.23	10.62
(10, 4)	3	15.26	17.08	4.73
(10, 4)	4	11.53	13.97	3.69

在与学术界 SOTA 的 RS 码实现 Cerasure 对比时（采用 Cerasure 原始测试方法以保持一致），FastTT 的编码吞吐在所有配置下均优于 Cerasure，平均提升 13.2%（最高 27.0%），如表2-3所示；解码吞吐平均提升 16.0%（最高 43.6%），如表2-4所示。其中仅在个别两块丢失配置下略低于 Cerasure（原因同上：两块丢失无法利用快路径），其余情形均全面领先，充分验证了优化方案在各类配置下的泛化性与有效性。

表 2-3 FastTT 与 Cerasure 的编码吞吐 (GB/s)

配置	FastTT	Cerasure
(6, 2)	47.97	44.20
(6, 3)	32.01	30.97
(8, 3)	32.75	28.72
(10, 4)	24.58	19.36

表 2-4 FastTT 与 Cerasure 的解码吞吐 (GB/s)

配置	丢失块数	FastTT	Cerasure
(6, 2)	1	88.41	76.22
(6, 2)	2	46.80	43.08
(6, 3)	1	87.69	75.45
(6, 3)	2	39.40	41.20
(6, 3)	3	21.09	17.82
(8, 3)	1	82.59	74.91
(8, 3)	2	39.28	39.94
(8, 3)	3	24.14	16.81
(10, 4)	1	73.95	69.89
(10, 4)	2	36.66	35.68
(10, 4)	3	21.59	15.60
(10, 4)	4	16.93	12.27

2.4.3 消融实验

为评估各项优化的独立贡献，本课题在 (8, 3) 配置下进行消融实验，结果如表2-5所示，解码吞吐为所有丢失情况的平均值。以原始 Two-tone 实现为基线：Erasure-Parity Mapping (EPM) 带来约 100.2% 的解码性能提升；在此基础上加入 Cache-Friendly Window Partitioning (CFWP) 使编解码吞吐进一步提升约 43.0%；再加入 Neighbor-Merging Access (NMA) 带来约 58.6% 的编码性能提升；最后 Traversal Pattern Selection (TPS) 贡献了约 43.5% 的解码吞吐提升。每项优化都对最终性能有清晰而显著的贡献。

表 2-5 各项优化技术的性能影响（GB/s）。CFWP：缓存友好窗口划分；NMA：相邻合并访问；EPM：统一解码抽象；TPS：遍历模式选择

CFWP	NMA	EPM	TPS	编码	解码
				13.16	7.54
		✓		13.16	16.85
✓		✓		23.32	24.10
✓	✓	✓		31.23	24.10
✓	✓	✓	✓	32.78	35.23

2.4.4 端到端存储系统实验

除上述基于内存（不含磁盘 I/O）的编解码吞吐实验外，本课题还进行了基于磁盘的端到端实验：利用 Ceph 的 vstart 搭建测试集群并创建纠删码存储池，再通过 rados 工具执行端到端 *PUT* 操作（将本地文件作为对象写入 Ceph）。由于 *PUT* 不仅包含编码过程，还包含较慢的磁盘写入，整体速度显著低于前述实验，但更能体现 FastTT 在生产级场景中的实际表现。结果（见表2-6）表明，FastTT 的端到端吞吐持续优于 ISA-L，平均提升 7.1%（最高 13.3%）。

表 2-6 Ceph 纠删码存储池的端到端 *PUT* 吞吐（MB/s）

配置	FastTT	ISA-L	Jerasure
(6, 2)	457.69	444.64	401.39
(6, 3)	450.03	397.06	362.83
(8, 3)	877.32	817.60	709.31
(10, 4)	573.04	546.44	490.08

综上，已完成的工作系统性地解决了 Two-tone 移位异或纠删码工程实现中的内存访问瓶颈，并在编码、解码、消融及端到端实验中全面验证了优化方案的有效性，达成了开题报告中设定的核心目标。

3 后期拟完成的研究工作及进度安排

本课题所定核心目标已基本完成，后期工作将从 2026 年 7 月持续到 2027 年 5 月，主要围绕已有优化结果的证据补强、小规模覆盖写下的增量更新验证以及硬

士毕业论文撰写展开，具体包括：

- 1) 持续整理和完善已完成的学术论文工作，补充必要实验，提升结论的完整性与说服力。
- 2) 深化对已有四项系统级优化（CFWP、NMA、EPM、TPS）生效机理的验证：在原有端到端吞吐与消融实验之外，补充直接刻画各优化点作用机制的细粒度实验，详见第一点说明。
- 3) 在现有 FastTT 编解码实现基础上，验证小规模覆盖写场景下的增量更新方法，详见第二点说明。
- 4) 系统整理研究成果，撰写硕士毕业论文并准备答辩。

3.1 第一点：已有系统优化的机理验证与实验深化

现有学术论文主要通过端到端吞吐量与消融（Ablation）实验，从结果层面验证了四项优化（缓存友好窗口划分 CFWP、相邻合并访问 NMA、统一解码抽象 EPM、遍历模式选择 TPS）带来的整体性能提升。后期拟在此基础上补充更直接刻画各优化点作用机制的细粒度实验，使“为何有效”与“有效多少”都得到充分支撑，具体规划如下。

缓存命中率验证（对应 CFWP）。缓存友好窗口划分的核心收益在于提升缓存局部性，但现有实验仅以吞吐间接反映。后期拟借助 perf 或 Cachegrind 等工具直接采集 L1/L2/LLC（末级缓存）的缓存命中率与缺失率（Cache Miss Rate），对比开启/关闭窗口划分及不同窗口大小 W 下的指标变化，直接证明缓存局部性的改善，并据此印证窗口大小取值的合理性。

内存访问量与写放大验证（对应 NMA）。相邻合并访问的理论收益是将校验块写入访问量约减半。后期拟通过访存计数（如 perf 内存事件或插桩统计）测量编码过程中对数据块、校验块的实际读写次数，验证其与理论复杂度 $\Theta((k + \frac{km}{2})l)$ 的吻合程度，量化写放大（Write Amplification）的降低幅度。

冗余访存消除验证（对应 EPM）。统一解码抽象通过将数据块解码与校验块修复融合，省去独立的修复阶段。后期拟对比统一解码流程与“解码+独立修复”两阶段流程的总访存次数与遍历轮数，量化被消除的冗余内存访问，直接解释 EPM 带来约一倍解码性能提升的来源。

单块丢失快路径验证（对应 TPS）。遍历模式选择针对最常见的单块丢失场景采用纵向单遍异或快路径。后期拟分别统计单块丢失与多块丢失场景下的访存模式与每码字访问次数，验证快路径“每码字恰好访问一次”的特性，并测量流式（Streaming）指令绕过缓存所带来的额外收益。

实验维度扩展。配合上述机理验证，进一步补充不同块大小、不同 (k, m) 配置与更多丢失模式下的细粒度数据，并对 (10, 4) 等不利配置下的性能瓶颈做针对性分析与优化探索。

3.2 第二点：小规模覆盖写下的 Two-tone/FastTT 增量更新

已有工作主要验证了 FastTT 在完整编码、完整解码和 Ceph 端到端 PUT 场景中的性能。后期拟进一步关注一种更具体的存储系统写入：已有对象中只有一小段数据被覆盖修改时，如何只更新受影响的数据段和校验段，而不是重新处理整条条带。

问题定位。在 Ceph 等纠删码存储系统中，RBD、CephFS 和 librados 可以对已有 RADOS 对象的指定偏移和长度写入数据。这类写入不是新建小对象或顺序追加，而是覆盖已有对象中的一小段内容。数据段被修改后，相应校验段必须同步更新；若每次都重新处理完整条带，未变化的数据也会参与计算和访存，容易带来额外开销。

技术思路。当一小段旧数据被新数据覆盖时，可先计算变化量 $\Delta = old_data \oplus new_data$ ，再把 Δ 并入对应校验段。传统 Reed-Solomon 码也能做增量更新，通常需要将 Δ 乘以有限域系数后更新校验段；Two-tone 码的更新位置由移位偏移决定，找到偏移后的区间后主要执行 XOR。因此，这一工作要验证的是：在同样支持增量更新的前提下，Two-tone/FastTT 能否凭借“偏移区间映射 + XOR”减少局部覆盖写中的计算和内存访问开销。

主要困难与挑战。后期工作的重点不是简单写出 $\Delta = old_data \oplus new_data$ ，而是把 Two-tone 的移位异或结构放到真实局部写和读-改-写（RMW）路径中验证。围绕这一研究方向，需要重点解决以下三项困难与挑战：

- **连续写入到移位后校验区间的映射与边界处理。**客户端只给出对象偏移和写入长度，后端却要定位到数据块、条带/窗口、校验块和移位后的校验偏移。一次小覆盖写可能跨越多类边界，因此需要拆成若干规范化更新任务，分别处理头尾非对齐区间和中间可向量化区间。
- **增量更新与完整重新编码的自适应选择。**增量更新处理范围小，但需要读旧数据、读旧校验并写回新数据和新校验；完整重新编码处理范围大，但访问更连续、SIMD 利用率可能更高。后期需结合更新大小、条带大小和 (k, m) 配置确定两条路径的切换条件。
- **瓶颈定位与 ISA-L Reed-Solomon 增量更新的公平比较。**ISA-L Reed-Solomon 也支持增量更新，不能将 Two-tone/FastTT 增量更新与 Reed-Solomon 完整重

新编码直接比较。后期需在纯计算层、RMW 读写层和可能的 Ceph 端到端层比较移位异或与有限域乘加的开销，明确适用边界。

具体工作进度安排为：

- 2026 年 7 月 – 2026 年 10 月：整理 FastTT 实现、Ceph 集成和已有实验结果，补充缓存命中率、实际访存次数等机理验证实验；同时实现小规模覆盖写增量更新原型，完成连续写入到移位后校验区间的基本映射与边界拆分，并与完整重新编码路径进行初步比较。
- 2026 年 11 月 – 2027 年 2 月：完善相邻区间合并、对齐和 SIMD XOR 等实现细节，补充与 ISA-L Reed-Solomon 增量更新的分层对比实验，整理不同写入范围与不同 (k, m) 配置下的切换条件和性能边界；若原型稳定且时间允许，开展 Ceph partial write 路径的小规模端到端验证。
- 2027 年 3 月 – 2027 年 5 月：完成硕士毕业论文主体撰写、图表整理、格式检查、定稿和答辩材料准备。

4 存在的困难与问题

本课题在推进过程中，主要面临以下困难与问题：

4.1 存在问题与困难

- 1) **特定配置下的性能瓶颈**：在 $(10, 4)$ 等配置中，第四个校验块需要 2 个码字的移位偏移，无法在 AVX2 的 256 位寄存器约束下与其他校验块合并，导致编码和解码性能相对下降，这是当前实现尚未完全克服的瓶颈。
- 2) **两块丢失场景的优化空间**：两块丢失时既无法使用单块丢失的直接异或快路径，又不具备高丢失数场景下异或占比高的特点，必须完整执行代入与消除，性能优势相对有限。
- 3) **优化方法论的通用性**：当前优化方案主要针对 Two-tone 码的特定移位结构，如何将其推广至更广泛的移位异或码族并形成通用方法论仍需深入研究。
- 4) **工程实现与系统集成的复杂性**：将算法以插件形式稳定集成进 Ceph 这类大型分布式系统，涉及接口适配、内存管理与正确性验证等诸多工程细节，调试与测试成本较高。
- 5) **小规模覆盖写的区间映射难点**：第二个创新点首先要把对象偏移和写入长度准确映射到数据块、条带单元、Two-tone 窗口和移位后的校验区间，避免跨边界和非对齐处理导致校验更新位置错误。

- 6) **增量更新策略的不确定性**：增量更新需要额外读取旧数据和旧校验，完整重新编码则访问更连续、SIMD 利用率可能更高；二者在不同更新大小和 (k, m) 配置下的切换条件仍需验证。
- 7) **公平比较与瓶颈归因问题**：ISA-L Reed-Solomon 本身也支持高效增量更新，需要区分性能差异来自有限域乘加、移位异或、内存访问，还是 Ceph 事务、网络 and 存储 I/O 等系统开销。

4.2 解决方案

针对上述困难，拟采取以下解决方案：

- 1) 针对寄存器约束导致的瓶颈，探索更灵活的码字分组与 SIMD 指令调度策略，或结合 AVX-512 等更宽寄存器进一步挖掘合并潜力。
- 2) 针对两块丢失场景，分析其访问模式特征，尝试设计介于单块快路径与通用消除之间的中间优化策略。
- 3) 通过抽象 Two-tone 优化技术中与具体偏移结构无关的共性部分，逐步构建面向移位异或码族的通用优化框架。
- 4) 充分利用 Ceph 官方的测试工具与社区文档，结合单元测试与端到端测试保证集成的正确性与稳定性。
- 5) 针对区间映射难点，设计统一的 range mapping 模块，将一次局部写转换为若干连续的规范化更新任务，分别处理边界区间和中间 SIMD XOR 主体区间。
- 6) 针对更新策略选择，通过不同更新大小、条带大小和 (k, m) 配置下的延迟、访存量和 CPU 周期实验寻找阈值，形成增量更新/完整重新编码的自适应选择策略。
- 7) 针对公平比较与瓶颈归因，按纯计算层、RMW 读写层和小规模系统层开展分层性能剖析，给出 Two-tone/FastTT 增量更新的适用边界。

5 如期完成全部论文工作的可能性

本课题目前进展顺利，已经完成了开题报告中规划的绝大部分核心工作：完成了 Two-tone 移位异或纠删码的原理分析与四项系统级优化方案的设计与实现，构建了高性能的 FastTT 纠删码库并成功集成至 Ceph 分布式存储系统，并通过编码、解码、消融以及端到端实验全面验证了优化方案的有效性，相关成果已整理形成学术论文投稿至 IPDPS 2026。后期主要工作集中在论文完善、优化机理验证、小规模覆盖写增量更新验证以及毕业论文撰写，工作目标明确、技术路线清晰，且

已有扎实的代码与实验基础作为支撑。综上所述，本课题能够按期完成全部论文工作，并有望取得高质量的研究成果。

参考文献

- [1] Ford D, Labelle F, Popovici F I, et al. Availability in Globally Distributed Storage Systems[C] //Proceedings of the USENIX Symposium on Operating Systems Design and Implementation. Vancouver, BC: USENIX Association, 2010.
- [2] Weatherspoon H, Kubiatowicz J D. Erasure Coding Vs. Replication: A Quantitative Comparison[C] //Proceedings of the Peer-to-Peer Systems. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002: 328-337.
- [3] Ghemawat S, Gobioff H, Leung S-T. The Google File System[C] //Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles. Bolton Landing NY USA: ACM, 2003: 29-43.
- [4] Calder B, Wang J, Ogus A, et al. Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency[C] //Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles. Cascais Portugal: ACM, 2011: 143-157.
- [5] Shvachko K, Kuang H, Radia S, et al. The Hadoop Distributed File System[C] //Proceedings of the IEEE Symposium on Mass Storage Systems and Technologies. Incline Village, NV, USA: IEEE, 2010: 1-10.
- [6] Weil S A, Brandt S A, Miller E L, et al. Ceph: A Scalable, High-Performance Distributed File System[C] //Proceedings of the USENIX Symposium on Operating Systems Design and Implementation. Seattle, WA: USENIX Association, 2006.
- [7] Reed I S, Solomon G. Polynomial Codes Over Certain Finite Fields[J]. Journal of the Society for Industrial and Applied Mathematics, 1960, 8(2): 300-304.
- [8] Luby M, Zuckermank D. An XOR-Based Erasure-Resilient Coding Scheme[J]. Tech Report, 1995.
- [9] Plank J S, Simmerman S, Schuman C D. Jerasure: A Library in C/C++ Facilitating Erasure Coding for Storage Applications - Version 1.2: CS-08-627[R]. Knoxville, TN 37996, USA: University of Tennessee, 2008.
- [10] Liu C, Wang Q, Chu X, et al. G-CRS: GPU Accelerated Cauchy Reed-Solomon Coding[J]. IEEE Transactions on Parallel and Distributed Systems, 2018, 29(7): 1484-1498.

- [11] Zhou T, Tian C. Fast Erasure Coding for Data Storage: A Comprehensive Study of the Acceleration Techniques[J]. ACM Transactions on Storage, 2020, 16(1): 1-24.
- [12] Uezato Y. Accelerating XOR-based Erasure Coding Using Program Optimization Techniques[C] // Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. St. Louis Missouri: ACM, 2021: 1-14.
- [13] Wang W, Lyu M, Niu T, et al. Fast Acceleration Strategies for XOR-Based Erasure Codes[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2025, 44(1): 331-344.
- [14] Gajalwar Y, Khilari S, Kulkarni H, et al. Exploring Data Protection with ISA-L: A Study on Erasure Coding[C] // Proceedings of the Asian Conference on Intelligent Technologies. KOLAR, India: IEEE, 2024: 1-6.
- [15] Gong X, Sung C W. Zigzag Decodable Codes: Linear-time Erasure Codes with Applications to Data Storage[J]. Journal of Computer and System Sciences, 2017, 89: 190-208.
- [16] Fu X, Wu C, Guo Y, et al. Two-Tone Shift-XOR Storage Codes[C] // Proceedings of the IEEE International Symposium on Information Theory. Melbourne, Australia: IEEE, 2021: 2996-3001.
- [17] Cheng X, Fu X, Guo Y, et al. Successively Solvable Shift-Add Systems — a Graphical Characterization[C] // Proceedings of the IEEE International Symposium on Information Theory. Melbourne, Australia: IEEE, 2021: 946-951.